

Software Pipeline Guide

From sensor data to dataset, cleaning, storage, real-time display and AI prediction

Written for the period when the **hardware is still being built**: the whole software is developed and demonstrated with public data standing in for the sensors, and is designed so that when the hardware is ready **only the data source changes**.

What you get	(1) this guide, (2) the code package classroom-energy-code.zip that implements every step, (3) a live dashboard.
Golden rule	The software only knows one data format (the 'sensor log': voltage, current, power per appliance + temperature, humidity, light). Public data and the ESP32 are just two ways of producing it.
Honesty rule	While the hardware is pending, results come from stand-in data . Say so in your report and demo. Re-run the same code on your own hardware dataset for the final results.

Verification status (what was actually tested): the full pipeline (steps 2, 3, training, the live server with replay, stream cleaning, gap handling, predictions and anomaly checks) was run end to end. The temperature file (UCI 374) was the **real** public file. The household power file (UCI 235) could not be downloaded in the test environment, so it was replaced by a **synthetic file written in exactly the documented UCI 235 format** (semicolon separated, '?' for missing). Therefore run step 2 on the real file yourself and tell us if any column name differs. Numbers shown in this guide that came from tests are marked **[test]**; they are NOT results on real data.

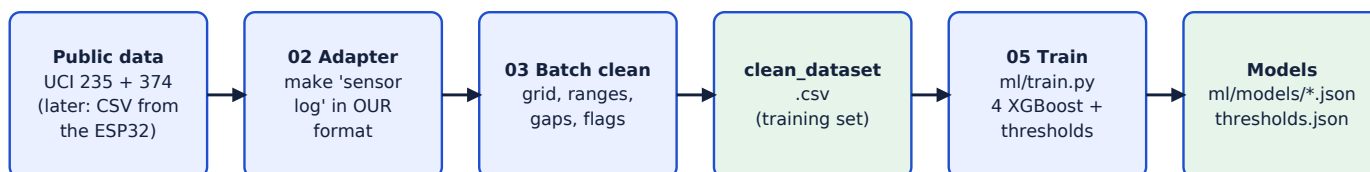
Contents

- **1** The big picture
- **2** Which sensor gives which data (the data dictionary)
- **3** The stand-in dataset while hardware is pending
- **4** Step 2: build the sensor log from the public data
- **5** Storage design (files and database)
- **6** Data cleaning and preprocessing, in detail
- **7** Feature engineering
- **8** Training the AI models and checking them
- **9** Real-time layer: ingest, clean, store, predict, show
- **10** Run everything, command by command
- **11** Switching to the real hardware
- **12** Troubleshooting, report checklist, viva questions

1. The Big Picture

There are two connected flows. The **offline flow** builds a dataset and trains the AI models. The **online flow** receives readings one by one, cleans them, stores them, predicts the next interval, checks for anomalies and shows everything on a dashboard. The trained models are the link between the two.

OFFLINE (build the dataset and train, run when you have new data)



ONLINE (real time, runs all the time)

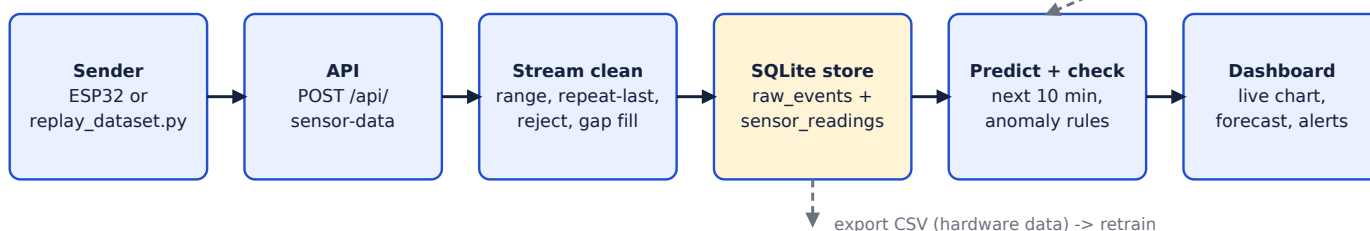


Figure 1: Both flows. Today the sender is a script that replays public data. Later it is the ESP32; nothing else changes.

What happens to one reading, in one sentence each

1. **Sensor** produces numbers (voltage, current, power per appliance, temperature, humidity, light).
2. **Sender** packs them into one JSON message every 10 minutes (virtual 10 minutes in our compressed demo).
3. **API** receives the message at POST /api/sensor-data.
4. **Raw store** keeps an untouched copy (table raw_events) so nothing is ever lost.
5. **Stream cleaning** checks ranges, repairs short holes, rejects unusable messages, fills short time gaps.
6. **Clean store** keeps the validated reading (table sensor_readings) with a quality flag.
7. **Features** are computed from recent history (lags, rolling averages, time of day, schedule).
8. **Models** predict total and per-appliance power for the next 10 minutes (and recursively for the next hour).
9. **Anomaly check** compares the next real reading with the prediction that was stored earlier.
10. **Dashboard** shows live values, actual vs predicted, forecast, data quality and the anomaly log.

Why this design is good for a capstone

- **Separation of concerns:** each stage is one script/module you can explain and test alone.
- **Same cleaning idea offline and online** (offline can look at the future, online cannot; Section 6.6 explains the difference).
- **Hardware independence:** a public dataset today, ESP32 tomorrow, same API, same database, same models.

2. Which Sensor Gives Which Data (Data Dictionary)

This table is the **contract** between hardware and software. Every column below exists in the database and in the dataset.

2.1 Raw measurements

Field	Unit	Source (hardware)	How it is obtained	Valid range	Used for
light_v, fan_v, proj_v	V	INA219 (one per appliance)	Bus voltage on the load side	0 - 30 (bench); 0 - 300 (stand-in data)	Sanity check, derive current, detect supply problems

Field	Unit	Source (hardware)	How it is obtained	Valid range	Used for
light_ma, fan_ma, proj_ma	mA	INA219	Current through the 0.1 ohm shunt	0 - 50,000 (bench realistically 0 - 1000)	Checks, derive power when W is missing
light_w, fan_w, proj_w	W	INA219 (V x I)	Averaged over the 10-second window (50 samples)	0 - 20,000	Main AI signal: targets and features
lux_raw	ADC 0-4095	LDR module on ESP32 pin 34	analogRead, averaged	0 - 4095	Is the room bright? Cross-check for lights ON
temp_c	deg C	DHT11	readTemperature()	-20 - 60	Context: fan use rises with heat
humidity	%	DHT11	readHumidity()	0 - 100	Context

2.2 Fields added by the software

Field	Meaning	How it is made
timestamp / vtime	The moment the reading belongs to (ISO 8601). The ESP32 has no real-time clock, so the hardware phase uses the virtual timetable clock; the stand-in data brings its own timestamps.	Replay: from the dataset. Simulation mode: scenario.py
total_w	Sum of the three appliance powers	light_w + fan_w + proj_w (recomputed by the server, never trusted from the sender)
energy_wh	Energy used in this 10-minute interval	total_w x 10 / 60. Divide by 1000 for kWh
schedule_flag	1 if the room should be in use (weekday, not a holiday, 09:00-16:00)	From the timetable function; a known fact, not measured
dq	Data-quality note: 'ok', 'imputed', 'gap', 'gap_filled', 'light_w:out_of_range', ...	Written by the cleaning code
source	'proxy' (public data), or hardware/simulation	Sent by the sender
fault_type, is_anomaly_injected	Ground-truth label, only in simulation mode (deliberate faults)	scenario.py; empty/0 for real and proxy data

2.3 Why each group matters to the AI

- **Per-appliance power** lets the AI forecast each appliance and name the one that misbehaves.
- **Voltage and current** are kept (not only watts) so you can validate the sensors, spot wiring problems, and show faculty that power = $V \times I$ was really measured.
- **Temperature and humidity** explain why a fan runs harder on a hot day. **Light level** helps detect lights left on in daylight.
- **Schedule** is the strongest context: the same 200 W is normal at 10:00 on Monday and a problem at 03:00 on Sunday.

3. The Stand-In Dataset While Hardware Is Pending

No single public dataset has fan, light and projector voltages together with room temperature. We therefore combine **two real public datasets** into one sensor log that has exactly the fields of Section 2.

	UCI 235: Individual household electric power consumption	UCI 374: Appliances energy prediction
Gives us	Voltage, current, 3 separate power channels ('sub-meters')	Room temperature and humidity (T1, RH_1) and 10-minute time steps
Rate	1 reading per minute	1 reading per 10 minutes (same as our design)
Size / period (per dataset page, verify)	about 2.07 million rows, Dec 2006 - Nov 2010	19,735 rows, 11 Jan - 27 May 2016 (137 days), no missing values

	UCI 235: Individual household electric power consumption	UCI 374: Appliances energy prediction
Gaps	about 1.25% of rows missing ('?'), including multi-hour outages: perfect for practising cleaning	none
File	household_power_consumption.txt (semicolon separated)	energydata_complete.csv

3.1 Column mapping (public data to our sensor log)

Our column	Public source	Transformation
light_w	Sub_metering_2 (laundry room: washer, dryer, fridge, light)	Wh per minute x 60 = watts; average over the 10-minute bin
fan_w	Sub_metering_3 (water heater + air conditioner: motor-like, biggest load)	same
proj_w	Sub_metering_1 (kitchen: dishwasher, oven, microwave: bursty on/off)	same
light_v, fan_v, proj_v	Voltage (one house voltage, about 240 V)	mean over the 10-minute bin; same value for all three
light_ma, fan_ma, proj_ma	(derived)	current = power / voltage x 1000
temp_c, humidity	UCI 374: T1 and RH_1	taken by step number (see 3.2)
lux_raw	not available	left empty (NaN): the pipeline must cope with a missing sensor

The mapping lives in one dictionary at the top of 02_build_proxy_dataset.py (CHANNEL_MAP). Change it and rebuild if you prefer another assignment.

3.2 Be clear about what this is

- The three sub-meters are **not** a fan, a light and a projector. They are household circuits used as stand-ins with the same data shape (three power channels with different behaviour).
- Temperature comes from a different house and year. The two files are joined **by step number**, not by real time, so temperature has no true physical link to the power. The model may learn little from it. That is fine for testing software, and must be stated in your report.
- Watts are in the hundreds (a house), while the bench will be 1-7 W. The AI does not care about scale; the dashboard has a scale factor for display only.
- Faculty-ready sentence: 'Public household data is used only to develop and validate the software pipeline; final results are produced from our own hardware dataset.'

3.3 Getting the files

```
python data_pipeline/01_download_datasets.py
```

- The script downloads both zips from archive.ics.uci.edu into data/raw/ (the household zip is roughly 20 MB; the text file is about 127 MB).
- Manual fallback:** open the dataset pages on archive.ics.uci.edu (search 'Individual household electric power consumption' and 'Appliances energy prediction'), download, unzip, and put household_power_consumption.txt and energydata_complete.csv into data/raw/.
- No internet at all? `python tools/make_test_fixtures.py --days 140 --also-appliances` writes FAKE files of the same format so you can test the software (never use them for results).

4. Step 2: Build the Sensor Log (the Adapter)

An **adapter** converts a foreign data source into our format. data_pipeline/02_build_proxy_dataset.py does it in six moves:

- Read UCI 374** to know how many 10-minute steps we need (19,735) and to take temperature and humidity.
- Choose a time window** in the household file: start 2007-01-11 00:00, same number of steps (it ends 2007-05-28). This window is chosen so a whole winter-spring season is covered.

3. **Read UCI 235** with `sep=';'` and `na_values=['?']`. The '?' marks become NaN (missing). Build a timestamp from the Date (dd/mm/yyyy) and Time columns.
4. **Convert units:** sub-meters are in **watt-hours per minute**; multiply by 60 to get **average watts** in that minute.
5. **Resample 1 minute to 10 minutes:** average the minutes inside each 10-minute bin, but only if at least 5 of the 10 minutes are valid. Otherwise the bin becomes NaN (a real gap). This rule is the honest way to handle missing minutes.
6. **Write the sensor log:** one row per 10 minutes with all columns of Section 2, current derived as $W / V \times 1000$, lux empty, source = 'proxy'.

```
python data_pipeline/02_build_proxy_dataset.py
# -> data/proxy/raw_sensor_log.csv (19,735 rows x 14 columns)
```

Key lines of the logic

```
hh[sub] = hh[sub] * 60.0 # Wh per minute -> watts
r = hh.set_index('ts')[cols].resample('10min')
mean, cnt = r.mean(), r.count()
mean = mean.where(cnt >= 5) # need >= 5 valid minutes per 10-min bin
out['fan_ma'] = out['fan_w'] / out['fan_v'] * 1000 # current from power and voltage
```

What the output looks like [test]: in a test run, 326 of 19,735 rows (1.7%) had no power values because a long outage covered them, and lux_raw was empty on every row. The cleaning step must handle both situations.

5. Storage Design

5.1 Four layers, each with one job

Layer	Where	Content	Why it exists
L0 Raw files	data/raw/*	Original public files, never edited	Reproducibility: you can always rebuild everything
L1 Sensor log	data/proxy/raw_sensor_log.csv	Our format, still dirty (holes, noise)	The same shape the hardware produces
L2 Clean dataset	data/processed/clean_dataset.csv	Complete 10-minute grid with quality flags	Training input; cleaning report documents every change
L3 Live database	data/classroom_energy.db (SQLite)	raw_events, sensor_readings, predictions, anomalies, kv	Real-time storage, dashboard queries, hardware data later exported as a CSV

5.2 Database tables

Table	Columns (main ones)	Purpose
raw_events	id, received_at, payload (the JSON text as received), accepted (0/1), reason	Audit trail. Rejected messages are kept too, with the reason
sensor_readings	id, vtime, light_v/ma/w, fan_v/ma/w, proj_v/ma/w, total_w, energy_wh, lux_raw, temp_c, humidity, schedule_flag, dq, source, fault_type, is_anomaly_injected, created_at	Clean readings; the AI reads history from here
predictions	id, made_at_reading_id, for_vtime, predicted_total_w, predicted_light_w, predicted_fan_w, predicted_proj_w, model_version	Stored BEFORE the real value arrives, so anomalies are checked fairly
anomalies	id, reading_id, detected_vtime, actual_w, expected_w, residual_w, severity_sigma, anomaly_type, likely_appliance, method, injected_label	Alert log shown on the dashboard
kv	key, value	Small settings and counters (auto cut-off, how many steps a channel was imputed in a row)

- **Why SQLite:** a single file, zero installation, enough for 10,000+ rows and one writer. Moving to PostgreSQL later only changes the connection code.
- **Index on vtime** makes 'find the prediction for this moment' instant.
- Export at any time: `http://localhost:8000/api/export.csv` gives the stored dataset as CSV (this is how hardware data becomes the training file).

6. Data Cleaning and Preprocessing, in Detail

Real sensor data is never perfect: wires come loose, Wi-Fi drops, power fails, values jump. If dirty data goes into the AI, the AI learns dirt. Cleaning is therefore documented, counted and reversible (raw data is always kept).

6.1 The eight batch-cleaning steps (script 03_clean_dataset.py)

#	Step	Rule	Why	What the report counts
1	Parse timestamps	Convert text to datetime; drop rows whose time cannot be read	A reading without a time is useless	bad timestamps dropped
2	Sort + de-duplicate	Sort by time; if a timestamp repeats keep the first	Network retries and file merges create duplicates and wrong order	duplicates dropped
3	Rebuild the 10-minute grid	Create every expected timestamp from first to last; missing ones become empty rows	Lag features ('value 1 day ago') need an unbroken clock	missing time steps added
4	Count missing values	Per column before any repair	Shows how bad the data was	missing per column
5	Range check	Voltage 0-300 V, current 0-50,000 mA, power 0-20,000 W, temp -20..60 C, humidity 0-100 %, lux 0-4095. Outside the range: set to missing	Negative watts or 999 degrees are sensor errors, not events	out-of-range per column
6	Frozen-sensor check	Voltage exactly identical for 36 steps (6 h): set to missing	A stuck sensor repeats one number; real voltage always wiggles	frozen values per column
7	Fill short gaps	A gap of at most 3 steps (30 min): linear interpolation between the neighbours. Longer gaps: left empty	Short holes are safe to bridge; long ones would invent behaviour (e.g. a whole evening)	values interpolated
8	Flag and derive	dq = ok / imputed / gap. Add total_w (NaN if any channel missing), energy_wh, schedule_flag, is_anomaly_injected = 0	Downstream code can drop or weight bad rows	ok / imputed / gap rows, longest gap, percent usable

6.2 The short-gap rule written out

```
def fill_short_gaps(s, max_fill):
    # s = one column
    isna = s.isna()
    run_id = (isna != isna.shift()).cumsum() # number each run of equal values
    run_len = isna.groupby(run_id).transform('sum') # length of the NaN run a row belongs to
    interp = s.interpolate(method='linear', limit_area='inside')
    return s.where(~(isna & (run_len <= max_fill)), interp) # only short runs are filled
```

A common mistake is `interpolate(limit=3)`: it fills the first three steps of a long gap and leaves the rest. The code above fills a gap **only if the whole gap is short**.

6.3 Example cleaning report [test]

Produced by deliberately corrupting a test sensor log (duplicates, shuffled order, 5 missing time steps, a negative power, 999 degrees, 41 frozen voltage values, scattered 1-2 step holes). It shows what the report looks like and that every defect was caught:

```
2_duplicate_timestamps_dropped: 2
3_missing_time_steps_added: 5
5_out_of_range_set_missing: {'light_w': 1, 'temp_c': 1}
6_frozen_voltage_set_missing: {'light_v': 41}
7_values_interpolated: 75
8_rows_ok: 19396 8_rows_imputed: 13 8_rows_gap_unusable: 326
8_longest_gap_steps: 280 8_percent_usable: 98.35
```

6.4 What is deliberately NOT cleaned away

- **Real spikes** (a big load switching on) are real energy events, not errors. We only remove impossible values. Removing real peaks would make the AI blind to exactly what we want to detect.

- **Scaling/normalising** is not applied because XGBoost (tree models) does not need it. An LSTM would need it (Section 8.7).
- **Log transform** of power is optional: power is right-skewed. Trees are insensitive to it, so it is skipped for simplicity.

6.5 What happens to rows that stay as gaps

They stay in the file with `dq = gap`, so the 10-minute clock remains continuous (lags stay correct). Feature building runs on the full grid, and only **afterwards** are rows with an empty target dropped for training: `F.dropna(subset=need)`. This is why the training log says 'Dropped 329 unusable rows' [test] instead of silently shifting the timeline.

6.6 Offline versus online cleaning (important difference)

Question	Batch (offline)	Stream (online, in the API)
Can it look at the next value?	Yes: interpolates between past and future neighbours	No, the future does not exist yet
Short gap repair	Linear interpolation	Repeat the last good value, at most 3 steps in a row; the 4th failure rejects the message
Out-of-range value	Set missing, then interpolate if short	Treat as missing, same repair; flagged in <code>dq</code> (e.g. 'light_w:out_of_range;light_w:imputed')
Missing time steps	Added to the grid as empty rows	Up to 6 steps are filled by repeating the last values (<code>dq = gap_filled</code>); longer: next row flagged 'gap_before:N_steps'
Duplicate / old timestamp	Dropped in step 2	Rejected with reason 'duplicate or out-of-order timestamp'
Where it lives	<code>data_pipeline/03_clean_dataset.py</code>	<code>backend/main.py: clean_reading()</code> and <code>sensor_data()</code>

Verified behaviours of the online cleaner [test]: a gap of 37 steps produced 3 imputed rows, 37 rejected messages (reason 'power missing for more than 3 steps') and a 'gap_before:37_steps' flag on the first row after the gap; a negative power plus 999 degrees was repaired and flagged; a payload with only voltage and current got its power computed ('from_v_and_i'); a repeated timestamp was rejected.

6.7 Pictures for the report

```
python data_pipeline/04_eda_figures.py
# data/processed/figures/: 1_missing_per_month.png, 2_daily_profile.png, 3_sample_week.png, 4_correlation.png
```

7. Feature Engineering

The model does not see raw rows; it sees **features**: numbers that describe the situation just before the moment we predict. `ml/features.py` builds them identically for training and for live prediction (one function, so the two can never disagree).

Problem definition: at time **t**, predict power at time **t+1** (the next 10 minutes) for total, light, fan and projector.

Group	Features	Formula / idea	Why
Current state	<code>total_w</code> , <code>light_w</code> , <code>fan_w</code> , <code>proj_w</code> , <code>lux_raw</code> , <code>temp_c</code> , <code>humidity</code>	values at time <code>t</code>	The best predictor of the next step is the present
Schedule	<code>schedule_flag</code> (now), <code>sched_next</code> (<code>t+1</code>)	timetable; <code>sched_next</code> is known in advance	Knowing a class starts in 10 minutes is legal information, not leakage
Cyclic time	<code>hour_sin</code> , <code>hour_cos</code> , <code>dow_sin</code> , <code>dow_cos</code> , <code>is_weekend</code>	$\sin/\cos$ of $2\pi \times \text{hour}/24$ and $\text{day}/7$, computed for the TARGET time <code>t+1</code>	23:50 and 00:00 are close; plain numbers 23 and 0 look far apart
Short lags	<code>lag_1</code> , <code>lag_2</code> , <code>lag_3</code> , <code>lag_6</code> , <code>lag_12</code>	<code>total_w</code> shifted by 1, 2, 3, 6, 12 steps	Momentum: 10, 20, 30, 60, 120 minutes ago
Seasonal lags	<code>lag_day</code> , <code>lag_week</code>	<code>total_w</code> shifted by 143 and 1007 steps = the same clock time as the target, one day / one week earlier	'What happened at this hour yesterday / last week'
Rolling stats	<code>roll_mean_6</code> , <code>roll_mean_12</code> , <code>roll_std_6</code> , <code>roll_mean_144</code>	mean / std of the last 1, 2 hours and 1 day	Level and volatility of recent usage

7.1 Leakage: the one thing that ruins results

- **Leakage** means using information that would not exist at prediction time. A feature may only use data up to time t (plus things known in advance, like the timetable).
- Never split train/test randomly: a random split puts neighbouring minutes in both sets and the test becomes a memory test. We split **chronologically**: first 70% train, next 15% validation (tuning, early stopping, thresholds), last 15% test (touched once).
- Example sizes [test]: 19,405 usable rows give about 13,583 train, 2,911 validation, 2,911 test.

7.2 Missing values in features

Lags at the start (nothing 1 week earlier yet) and the empty `lux_raw` are NaN. XGBoost understands NaN natively (it learns which branch to send missing values to), so no imputation is needed. Linear Regression and Random Forest (used only as comparison) get NaN replaced by 0.

8. Training the AI Models and Checking Them

Command: `python ml/train.py --csv data/processed/clean_dataset.csv` (or without `--csv` to train from the live database, which is what you will do with hardware data).

8.1 What gets trained

Model	Role	Notes
Naive baseline	next value = current value	The score every real model must beat. If you cannot beat it, the AI is not adding value
Linear Regression	simple comparison	Cannot capture on/off and schedule effects
Random Forest	comparison	Many trees averaged; slower; can overfit
XGBoost (production)	4 models: total, light, fan, projector	Trees added one by one, each fixing the previous errors; fast (milliseconds); handles NaN; saved as <code>ml/models/xgb_*.json</code>

8.2 The 'delta' trick

Instead of predicting the next watts directly, the model predicts the **change** from now: $\text{target} = \text{value}(t+1) - \text{value}(t)$, and the code adds the current value back. The model then only has to learn what changes, which is much easier and made XGBoost clearly better than before in our tests. Predictions are clipped at 0 W because power cannot be negative.

8.3 Training procedure (what the script does)

1. Load the dataset; drop rows where the measurements or the target are empty (gap rows).
2. Build features, create targets, split chronologically 70/15/15.
3. Train XGBoost on the training part with **early stopping**: stop adding trees when the validation error has not improved for 30 rounds (prevents overfitting).
4. Compute residuals (actual minus predicted) on the **validation** set: their mean and standard deviation become the anomaly thresholds (`thresholds.json`, one entry per target).
5. Evaluate every model once on the untouched **test** set and print the comparison table.
6. Save the four XGBoost models and the thresholds; the running server reloads them automatically.

8.4 Reading the metrics

Metric	Meaning	Good when
MAE (W)	Average absolute error: mean of $ \text{actual} - \text{predicted} $	Small; easy to explain ('off by 90 W on average')
RMSE (W)	Square root of the mean squared error; punishes big misses	Small; compare with MAE: much larger means occasional big misses
R-squared	Share of variation explained (1 = perfect, 0 = no better than the average, negative = worse)	Close to 1; must beat the naive baseline

Example output format [test, fake household + real temperature, so numbers are NOT real results]:

```
TOTAL POWER, next 10 minutes (TEST set)
Model      MAE (W)  RMSE (W)  R2
Naive (next = now)  183.380  231.818  -0.042
Linear Regression  138.476  175.160   0.405
```

Random Forest	135.300	169.868	0.440
XGBoost (best iter 108)	133.681	168.384	0.450

8.5 Anomaly detection

- **Residual rule:** at time $t+1$, residual = actual total - predicted total. Convert to a z-score: $z = (\text{residual} - \text{mean_val}) / \text{std_val}$. If $|z| > 3$, raise an anomaly (over- or under-consumption). This adapts to hour, day and schedule because the prediction already knows what is normal.
- **Culprit appliance:** repeat the z-score for light, fan and projector residuals; the largest $|z|$ names the appliance.
- **Waste rule** (classroom mode): power above 0.3 W while schedule_flag = 0. It catches long-running faults that the residual rule 'gets used to' (once a fault persists, the lag features make it look normal). In simulation tests the combined rules reached recall 1.00 while the residual rule alone missed many. This rule is switched off for the household stand-in data, because a house is legitimately used at night.
- **Without labels** (stand-in/real data) precision and recall cannot be computed; the script prints the share of rows flagged, and you inspect the flagged rows by hand. With the simulation mode's injected faults (about 3% labelled) you can report precision, recall and F1.

8.6 Honest reporting rules

- Report results only from the **test split**, and say the data source (stand-in or hardware).
- Always show the naive baseline next to the model. In a small-data test (3,000 rows) XGBoost did not beat 'next = now'; with about 8,500 rows it did clearly [test, simulated data]. More data and a good split matter.
- Do not tune on the test set. Do not quote simulated-data numbers as hardware results.

8.7 Optional extension: LSTM

An LSTM (a neural network that reads the last 12 steps as a sequence) can be added later with TensorFlow. It needs scaled inputs (for example MinMaxScaler fitted on the training set only) and takes minutes to train. Use it only as a comparison; XGBoost stays the production model because it is faster and simpler to run live.

9. Real-Time Layer: Ingest, Clean, Store, Predict, Show

9.1 The message (what the sender posts every 10 minutes)

```
POST /api/sensor-data
{ "timestamp": "2007-01-23T11:50:00", "source": "proxy",
  "light_v": 237.5, "light_ma": 252.6, "light_w": 60.0,
  "fan_v": 237.5, "fan_ma": 1591.4, "fan_w": 378.0,
  "proj_v": 237.5, "proj_ma": 454.7, "proj_w": 108.0,
  "lux_raw": null, "temp_c": 17.8, "humidity": 37.6 }
```

Reply:

```
{ "accepted": true, "reading_id": 366, "dq": "ok",
  "predicted_next_w": 362.1, "predicted": { "total_w": 362.1, "light_w": 39.5, "fan_w": 282.3, "proj_w": 47.3 },
  "anomaly": false, "severity_sigma": 0.4, "culprit": null, "method": null, "command": null }
```

Fields may be null; the cleaner handles them. If a sender has no timestamp (the simulation mode), the built-in virtual timetable provides time and the reply carries a command (light %, fan %, projector) for the hardware to apply.

9.2 Prediction timing

The prediction is stored BEFORE the real value is known, so the check is honest.

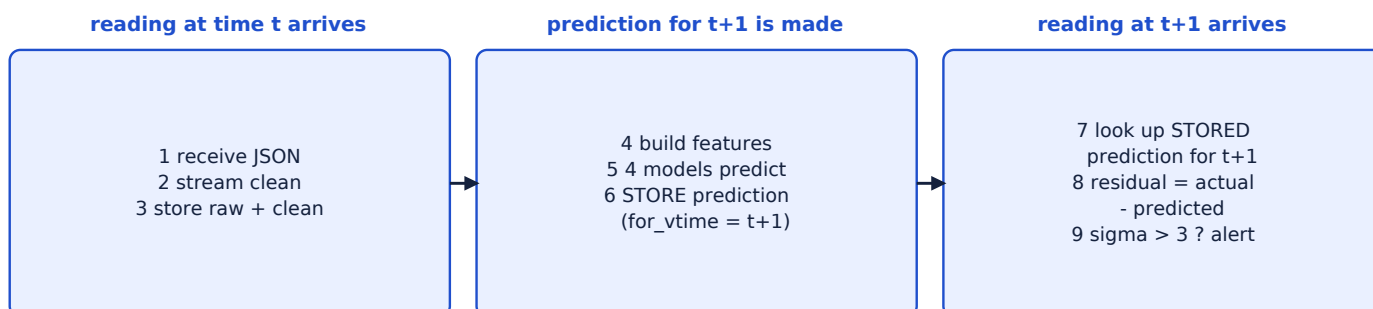


Figure 2: The prediction for $t+1$ is stored when t arrives and compared when $t+1$ arrives.

9.3 API endpoints

Endpoint	Method	Returns / does
/api/sensor-data	POST	Ingest + clean + store + predict + anomaly check (Section 9.1)
/api/current	GET	Latest clean reading, latest prediction, energy of the last 144 steps and equivalent cost, model status
/api/history?limit=200	GET	Last N readings with the prediction that was made for each (for the actual-vs-predicted chart)
/api/prediction?steps=6	GET	Recursive forecast: steps x 10 minutes ahead (6 = next hour) with total and per appliance, plus forecast energy in Wh. Up to 144 steps (1 day, takes about 3 seconds)
/api/anomalies	GET	Anomaly log, optional filter ?appliance=fan
/api/quality	GET	Share of clean readings, accepted/rejected counts, top rejection reasons
/api/control	POST	{'auto_cutoff': true}: in simulation mode a detected anomaly makes the next command switch loads off
/api/export.csv	GET	The whole stored dataset as CSV

9.4 Dashboard panels

- Cards: total, light, fan, projector power; voltage; room temperature; predicted next 10 minutes; forecast next hour; data quality; status (Normal / ANOMALY); equivalent energy and cost.
- Chart 1: actual total power vs predicted (the line you show faculty). Chart 2: light, fan and projector power.
- Anomaly log: time, actual vs expected, sigma, culprit appliance, rule that fired, and whether it was an injected fault.
- The page refreshes every 5 seconds from the API, so it works the same with replayed data and with the ESP32.

9.5 Making the stand-in data behave like a live sensor

```
python tools/replay_dataset.py --start 15000 --limit 600 --rate 5
# --rate 5 = 5 readings per second (a virtual day in about 29 seconds)
# --rate 0 = as fast as possible (about 40 readings per second [test])
```

The replay tool sends rows **with their holes** (null values), so the in-stream cleaner is exercised exactly as it will be with a real device.

10. Run Everything, Command by Command

#	Command (project folder, venv active)	What it does	Expect
0	<code>python -m venv venv venv\Scripts\activate pip install -r requirements.txt</code>	Installs Python packages	No errors
1	<code>python data_pipeline/01_download_datasets.py</code>	Downloads the two public datasets	Two files in data/raw/
2	<code>python data_pipeline/02_build_proxy_dataset.py</code>	Builds data/proxy/raw_sensor_log.csv	19,735 rows x 14 columns; missing counts printed
3	<code>python data_pipeline/03_clean_dataset.py</code>	Cleans; writes clean_dataset.csv and cleaning_report.json	Report with 8 numbered lines
4	<code>python data_pipeline/04_eda_figures.py</code>	Four report pictures	PNG files in data/processed/figures/
5	<code>python ml/train.py --csv data/processed/clean_dataset.csv</code>	Trains 4 XGBoost models, thresholds, comparison table	Table with Naive / LR / RF / XGBoost; models in ml/models/
6	<code>uvicorn backend.main:app --host 0.0.0.0 --port 8000</code>	Starts the server (terminal 1)	Open http://localhost:8000
7	<code>python tools/replay_dataset.py --start 15000 --limit 600 --rate 5</code>	Streams data like the ESP32 (terminal 2)	Dashboard fills; predictions appear; 'models_ready' true

A good first run order

1. Do steps 0 to 5 once. Look at the cleaning report and the model table; save screenshots.
2. Start the server and replay a quiet section first (`--start 15000`), then a section that crosses a data outage to show the data-quality card, rejections and gap handling.
3. Open <http://localhost:8000/api/quality> and [/api/anomalies](http://localhost:8000/api/anomalies) in the browser to show the raw JSON.
4. From a phone on the same Wi-Fi open http://LAPTOP_IP:8000 to show the live dashboard on another device.

Timing guide [test]: the replay handled about 40 readings per second with models loaded, so the full 19,735-row log takes about 8-10 minutes. The 144-step forecast took about 3 seconds.

Simulation mode (no dataset)

`python tools/fake_esp32.py --steps 3000 --fast` uses the virtual classroom timetable and injects labelled faults. It is the only mode in which precision/recall of the anomaly detector can be measured. The dashboard's red buttons force a fault for a live demo.

11. Switching to the Real Hardware

Part	Stand-in phase (now)	Hardware phase (later)	Code change
Sender	<code>tools/replay_dataset.py</code>	ESP32 firmware (firmware/esp32_energy)	None in backend; flash firmware
Voltage, current, power	From public sub-meters	From the three INA219 sensors; the firmware now sends <code>_v</code> , <code>_ma</code> and <code>_w</code>	None
Temperature, humidity, light	UCI 374 temperature; no light	DHT11 and LDR	None
Time	Dataset timestamps	Virtual timetable (simulation mode) or timestamps if you add an RTC/NTP	None
Training data	<code>clean_dataset.csv</code> from the adapter	<code>/api/export.csv</code> from the live database	<code>python ml/train.py</code> (reads the database)
Waste rule	Off (source = 'proxy')	On (classroom schedule)	None (automatic)

Checklist for the day the hardware works

1. Stop the server. Delete `data/classroom_energy.db` and `ml/models/*.json` (so no stand-in rows or models remain).
2. Flash the ESP32 firmware, start the server, confirm rows with `dq = ok` arrive.
3. Collect about 24 hours (about 8,600 rows in simulation mode).

- Run `python ml/train.py`; compare with the stand-in results in one report slide ('pipeline validated on public data, results reproduced on our hardware').
- Re-run `03_clean_dataset.py --input exported.csv` if you want the batch cleaning report for the hardware dataset.

12. Troubleshooting, Report Checklist, Viva Questions

Problem	Cause	Fix
Step 1 download fails	Firewall / no internet	Manual download (Section 3.3) into data/raw/
Step 2: KeyError for a column	A file version has different column names	Open the file, compare names with Section 3.1, adjust CHANNEL_MAP / column names at the top of the script
Step 2 is slow	The household file has 2 million rows	Normal: about a minute
Training says 'Need at least ~600 rows'	Too little data	Use the full clean dataset or collect more
Dashboard shows 'no model yet'	You have not run train.py	Run step 5; the server reloads models automatically
Replay: connection refused	Server not running or wrong port	Start uvicorn first; --url http://127.0.0.1:8000
Replay: many rejected	Data outage in that region	Expected: see /api/quality for reasons. Choose another --start
Anomaly log is huge on stand-in data	Household behaviour is irregular; 3-sigma on heavy-tailed errors	Normal for a house; the point is the machinery. Use simulation mode for labelled tests
Pandas/XGBoost dtype error	Empty columns arrive as text	Already handled in predict.py (numeric coercion); keep packages up to date

Report checklist

- Problem statement, data dictionary (Section 2), architecture figure (Figure 1).
- Dataset section: which public data, mapping table, the honesty paragraph (Section 3.2).
- Cleaning section: the eight steps, the cleaning report numbers, 2-3 figures from step 4.
- Feature table, chronological split, leakage explanation.
- Results: comparison table with the naive baseline; anomaly results; limitations.
- Real-time demo: dashboard screenshots, API JSON, the data-quality card.

Questions faculty may ask (and where the answer is)

Question	Short answer
Why not just a threshold like 'alert above 500 W'?	Normal power varies by hour and schedule; the model predicts what is normal now and alerts on the deviation (Section 8.5).
Why a chronological split?	Random splits leak the future through lag features and inflate scores (Section 7.1).
Why is XGBoost the production model?	Accurate, millisecond inference, handles missing values, small data friendly (Section 8.1).
Is your result real?	Stand-in data validates the software; final numbers come from our hardware dataset (Sections 3.2, 11).
How do you handle missing or wrong sensor data?	Raw copy kept, ranges checked, short gaps repaired, long gaps flagged and excluded, everything counted (Section 6).
How does it become real time?	Every reading triggers clean, store, features, prediction; the next reading is compared with the stored prediction (Section 9.2).
What changes with real hardware?	Only the sender and the training file (Section 11).